

Rajalakshmi Engineering College

Name: Sujit S
Email: 240701545@rajalakshmi.edu.in
Roll no: 240701545
Phone: 9444081254
Branch: REC
Department: CSE - Section 7
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 3_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Nikila is working as an intern in a software firm and is practicing with a matrix where each row represents a set of numerical values. Her task is to identify the row with the highest sum of its elements and remove that row from the matrix. After removing the row with the highest sum, Nikila needs to print the updated matrix.

Your task is to help Nikila in implementing the same. If there are two or more rows that have same the highest sum, the firstly encountered row is deleted.

Input Format

The first line of the input consists of two space-separated integers, R and C, representing the number of rows and columns in the matrix, respectively.

The following R lines each contain, C space-separated integers representing the matrix elements.

Output Format

The output prints the matrix after removing the row with the highest sum. Each row should be printed on a new line, with elements separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2 2

1 2

3 4

Output: 1 2

Answer

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int R = sc.nextInt();
        int C = sc.nextInt();
        int[][] matrix = new int[R][C];
        for (int i = 0; i < R; i++) {
            for (int j = 0; j < C; j++) {
                matrix[i][j] = sc.nextInt();
            }
        }
        int maxSum = Integer.MIN_VALUE;
        int maxIndex = 0;
        for (int i = 0; i < R; i++) {
            int rowSum = 0;
            for (int j = 0; j < C; j++) {
                rowSum += matrix[i][j];
            }
            if (rowSum > maxSum) {
                maxSum = rowSum;
                maxIndex = i;
            }
        }
        for (int i = 0; i < R; i++) {
            if (i != maxIndex) {
                for (int j = 0; j < C; j++) {
                    System.out.print(matrix[i][j] + " ");
                }
                System.out.println();
            }
        }
    }
}
```

```

    }
    }
    for (int i = 0; i < R; i++) {
        if (i == maxIndex) continue;
        for (int j = 0; j < C; j++) {
            System.out.print(matrix[i][j] + " ");
        }
        System.out.println();
    }
}
}
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Emma is a data analyst working with a grid-based system where each cell contains important numerical data. The grid represents spatial data, inventory records, or structured reports that require periodic updates.

Due to system updates and new requirements, Emma needs to modify the grid in the following ways:

She wants to insert either a new row or a new column at a given position. Later, she needs to delete either a row or a column from the modified matrix.

Input Format

The first line contains two integers rows and cols (the dimensions of the matrix).

The next rows lines contain cols space-separated integers representing the initial matrix.

The next line contains two integers insertType and insertIndex:

- insertType = 0 for row insertion, 1 for column insertion.
- insertIndex is the position where the new row/column should be added.

If inserting a row, the next cols integers represent the new row or If inserting a

column, the next rows integers represent the new column.

The next line contains two integers deleteType and deleteIndex:

- deleteType = 0 for row deletion, 1 for column deletion.
- deleteIndex is the position to be deleted.

Output Format

The first line of output prints the string "After insertion" followed by the modified matrix with the inserted row or column.

Each row of the matrix is printed on a new line with space-separated integers.

The next line prints the string "After deletion" followed by the final matrix after the specified deletion operation.

Each row of the resulting matrix is printed on a new line with space-separated integers.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3 3

1 2 3

4 5 6

7 8 9

0 1

10 11 12

1 2

Output: After insertion

1 2 3

10 11 12

4 5 6

7 8 9

After deletion

1 2

10 11

4 5

7 8

Answer

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner S = new Scanner(System.in);
        int rows = S.nextInt();
        int cols = S.nextInt();
        List<List<Integer>> matrix = new ArrayList<>();
        for (int i = 0; i < rows; i++) {
            List<Integer> row = new ArrayList<>();
            for (int j = 0; j < cols; j++) {
                row.add(S.nextInt());
            }
            matrix.add(row);
        }
        int insertType = S.nextInt();
        int insertIndex = S.nextInt();

        if (insertType == 0) {
            List<Integer> newRow = new ArrayList<>();
            for (int j = 0; j < cols; j++) {
                newRow.add(S.nextInt());
            }
            matrix.add(insertIndex, newRow);
            rows++;
        } else {
            for (int i = 0; i < rows; i++) {
                int val = S.nextInt();
                matrix.get(i).add(insertIndex, val);
            }
            cols++;
        }
        System.out.println("After insertion");
        printMatrix(matrix);
        int deleteType = S.nextInt();
        int deleteIndex = S.nextInt();

        if (deleteType == 0) {
            matrix.remove(deleteIndex);
            rows--;
        } else {
            for (int i = 0; i < rows; i++) {
```

```

        matrix.get(i).remove(deleteIndex);
    }
    cols--;
}
System.out.println("After deletion");
printMatrix(matrix);
}
private static void printMatrix(List<List<Integer>> matrix) {
    for (List<Integer> row : matrix) {
        for (int val : row) {
            System.out.print(val + " ");
        }
        System.out.println();
    }
}
}
}
}

```

Status : Correct

Marks : 10/10

3. Problem Statement:

Emma, a budding computer vision enthusiast, is working on a challenging image processing project. She has a square image represented as a 2D matrix of integers. As part of a special filter operation, she needs to rotate the image by 90 degrees clockwise, but there's a twist — she must perform the rotation in-place, using no extra space.

This means Emma has to rotate the matrix without creating a new one. Your task is to help her implement a Java program that takes this square matrix as input and rotates it within the same structure.

Can you help Emma efficiently rotate the image so that her project can move to the next stage?

Input Format

The first line of input contains a single integer n , representing the number of rows and columns of the square matrix (i.e., the matrix is of size $n \times n$).

The next n lines each contain n space-separated integers, representing the

elements of each row of the 2D array.

Output Format

The first line of output prints "Rotated 2D Array:"

The next n lines of output print the rotated matrix.

Each line contains n space-separated integers representing a row of the rotated matrix.

Refer to the sample output for format specification.

Sample Test Case

Input: 3

1 2 3

4 5 6

7 8 9

Output: Rotated 2D Array:

7 4 1

8 5 2

9 6 3

Answer

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[][] mat = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                mat[i][j] = sc.nextInt();
            }
        }
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                int temp = mat[i][j];
                mat[i][j] = mat[j][i];
                mat[j][i] = temp;
            }
        }
    }
}
```

```

    }
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n / 2; j++) {
            int temp = mat[i][j];
            mat[i][j] = mat[i][n - 1 - j];
            mat[i][n - 1 - j] = temp;
        }
    }
    System.out.println("Rotated 2D Array:");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            System.out.print(mat[i][j] + " ");
        }
        System.out.println();
    }
}
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Alex is a treasure hunter who collects valuable items during their quests. Each item has a specific point value, and Alex wants to maximize their score by strategically removing items one at a time.

The rule is simple: Alex removes the item with the highest point value in each step until no items are left, summing the values of the removed items to calculate the maximum score.

Help Alex to complete his task.

Input Format

The first line of input consists of an integer N, representing the size of the array.

The second line of input consists of N space-separated integers, representing the point values of the items.

Output Format

The output prints "Maximum Sum: " followed by the calculated maximum score after removing all items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 14

7 14 21 28 35 42 49 56 63 70 77 84 91 98

Output: Maximum Sum: 735

Answer

```
import java.util.*;
class main{
    public static void main(String args[]){
        Scanner S=new Scanner(System.in);
        int n=S.nextInt();
        int s=0;
        for(int i=0;i<n;i++){
            s+=S.nextInt();
        }
        System.out.print("Maximum Sum: "+s);
    }
}
```

Status : Correct

Marks : 10/10